SIS 4: Refactor Your App to MVVM

ViewModel · StateFlow · Unidirectional data flow · Covers Lectures 1–6

	Details
Work	Individual
Starting point	Your SIS3 app (same repository, new commits)
Final deadline	13.10.2026
Submit	Link to your GitHub repository in the Teams assignment
Defense	In class after the deadline, 5–7 minutes per student
AI tools	Allowed (same rules as SIS3)

1. Why this assignment matters

Almost every real Android codebase uses MVVM, and almost every Android job interview asks about it: "What is a ViewModel? Why does it survive rotation? What is StateFlow?" After this assignment, you will be able to answer from your own experience, not from memorized definitions.

- **It fixes real bugs.** Right now, if you rotate your phone, some state in your app is probably lost. After this refactor, it isn't.
- **It prepares your app for the next lectures.** Next we connect to the internet and a database. If your architecture is right, only the Repository changes. Your screens will not even notice.
- **It makes your code testable and previewable.** A stateless screen can be previewed in every state (loading, error, empty) without running the app.

> **Remember the restaurant**
>
> View = the waiter: shows things, passes orders, never cooks.
>
> ViewModel = the chef: takes orders, prepares the state, survives rotation.
>
> Model / Repository = the fridge: stores the data, knows where it comes from.
>
> **Events go up to the ViewModel. State comes down to the screen.**

2. Part A: Plan before code (10 pts)

Before refactoring, draw the architecture of your app (the restaurant picture, with your own names). Put it in /design and commit it first.

- **Screens (View):** which screens your app has.
- **ViewModels:** which ViewModel serves which screen.
- **Order board:** the fields of each UiState (for example items, searchQuery, selectedCategory, isLoading, error).

- **Events:** the user actions (for example onSearchChange, onCategorySelected, onFavouriteClick, onRetry).
- **Repository:** where the data lives.

Also write a short **"trace one click"** paragraph: when the user taps something, which function is called, what changes in the state, and what the screen shows.

3. Part B: Implementation (70 pts)

3.1 Data layer (15 pts)

- A **Repository** class owns the data. The UI never touches the data list directly.
- The repository behaves like a real server: it waits with **delay()** before returning data.
- It can **simulate an error** (for example a flag shouldFail = true), so you can test what the user sees when something goes wrong.

3.2 ViewModel and state (25 pts)

- A **ViewModel** for your main screen, with a **UiState data class** (only val properties).
- A private **MutableStateFlow** and a public read-only **StateFlow**. State is updated with **copy()**.
- The UiState covers **four situations**: loading, content, empty (no results), and error.
- At least **3 user actions** handled in the ViewModel. Choose what fits your app: search by text, filter by category, sort, toggle favourite, retry after error.
- All important state **survives rotation**: search text, filter, favourites.

3.3 Stateless screens (20 pts)

- Each screen composable receives **uiState** and **lambdas** (events) and **does not know the ViewModel exists**.
- A small **...Route** composable connects the ViewModel and the screen using **collectAsStateWithLifecycle()**.
- **No business logic** (filtering, sorting, searching) inside composables.
- A **@Preview for each of the four states** (loading, content, empty, error), using fake UiState objects.

3.4 Proof it works (10 pts)

- Add **Log.d** to the Activity lifecycle methods (onCreate, onStart, onResume, onPause, onStop, onDestroy) and to the ViewModel (init and onCleared).
- In the README, paste the **Logcat output after rotation** and explain in 3–4 sentences, in your own words, why the Activity was recreated but the ViewModel was not.

4. Part C: README, AI log and Git (20 pts)

- **README architecture section (10 pts):** the updated diagram from Part A and the "trace one click" explanation, now with real file and function names.
- **AI_USAGE.md (5 pts):** same questions as SIS3, plus: **"Find one place where AI put logic in the wrong layer (for example filtering inside a composable), and explain how you moved it."**
- **Git (5 pts):** meaningful commits

5. Check yourself

> **The preview test**
>
> Can you see your error state in a Preview, without running the app? If not, your screen still depends on the ViewModel.

> **The rotation test**
>
> Type a search, choose a filter, mark a favourite, then rotate the phone. Is everything still there? If not, some state is still living in the screen.

> **Is remember forbidden now?**
>
> No. Use this rule:
>
> If the user would be annoyed to lose it on rotation, or it is data, it goes in the **ViewModel**.
>
> If it is only how the UI looks right now (an animation, whether a dropdown is open), **remember** is fine.

6. Defense: your points count only after it

Same rule as SIS3: all points become valid **only after you pass the defense**.

What happens (5–7 minutes)

- **Demo (1 min):** show your app, rotate the phone, and show that the state survives. Show the error state.
- **Live change (2–3 min):** a small feature that goes through the layers, for example "add sorting by name", "show the number of results in the TopAppBar", "add a Retry button to the error state". No AI; autocomplete and documentation are allowed.
- **Questions (2 min):** for example "Why is _uiState private?", "Rotate the phone: what survived and why?", "Show me the path of a click through your files.", "Why can you preview the error state now?"

Outcomes

Result	What it means
Passed	You can explain your architecture and make the change. **All your points count.**
Partly passed	You understand most of it but cannot explain a specific part. **The points for that part are removed**; the rest counts.
Not passed	You cannot explain your own code or make a basic change. **The work is not accepted and receives 0 points.**

7. Grading

Total: 100 points + up to 10 bonus. Every row counts only after the defense.

Part	What is checked	Points
A. Architecture plan	Diagram, UiState fields, events, "trace one click", committed first	10
B1. Data layer	Repository owns data, delay(), simulated error	15
B2. ViewModel and state	UiState, private MutableStateFlow / public StateFlow, copy(), 4 states, 3+ actions, survives rotation	25
B3. Stateless screens	uiState + lambdas, Route with collectAsStateWithLifecycle, no logic in composables, 4 state previews	20
B4. Proof	Lifecycle and ViewModel logs, rotation explanation in README	10
C. README, AI log, Git	Architecture section, AI_USAGE.md with "wrong layer" example, commits over time	20
Total		**100**

Bonus (up to +10)

- A domain layer with at least two **use cases** (the recipe book from the lecture): +5
- A detail screen with its own ViewModel that gets the id through **SavedStateHandle**: +5
- A **unit test** for one ViewModel function: +5

8. Checklist before you submit

- ☐ /design: architecture diagram, UiState fields, events, "trace one click" (first commit)
- ☐ Repository owns the data, with delay() and a simulated error
- ☐ ViewModel with a UiState data class (only val)
- ☐ Private MutableStateFlow, public StateFlow, updates with copy()
- ☐ Loading, content, empty and error states
- ☐ At least 3 user actions handled in the ViewModel
- ☐ Screens receive uiState + lambdas and don't know the ViewModel
- ☐ Route composable with collectAsStateWithLifecycle()
- ☐ No filtering, sorting or searching inside composables
- ☐ Previews for loading, content, empty and error
- ☐ Rotation test passes (search, filter, favourites survive)
- ☐ Lifecycle + ViewModel logs; rotation explanation in README
- ☐ README architecture section with real file and function names
- ☐ AI_USAGE.md with a "wrong layer" example
- ☐ I can trace one click through my files and explain every layer